In [ ]:
```
#COSC 522 UTK
#Project 1
#Student Name: Dylan Shaffer
#AI Statement: The only form of AI used in this project is the use of autocomplete
```

In [ ]:
```

```

In [ ]:
```
#Load in data
import pandas as pd

url = 'https://gist.github.com/rhema/3b4b729d98978b9020d85c9b9e7c9dd6/raw/e0831e961
#url = 'C:\\Users\\Dylan\\Downloads\\tempfile.csv'
all_data = pd.read_csv(url)
```

In [ ]:
```
print(all_data.tail())
print(len(all_data))
```

In [ ]:
```
train_data = all_data.iloc[:275,:]
test_data = all_data.iloc[275:,:]
print(len(test_data))
```

In [ ]:
```
## Code for buildling the classifier
## Train only with the "train_data"

#Following section based off of live coding from 2/23 lecture
#Type Section
probability_type_count = {}
for row in train_data.iterrows():
    tt = row[1]['Type']
    if tt in probability_type_count:
        probability_type_count[tt] += 1
    else:
        probability_type_count[tt] = 1
print("Type Count: ", probability_type_count)

type_total = 0.0
for i in probability_type_count.keys():
    type_total += probability_type_count[i]

prior_probability_type = {}
for i in probability_type_count.keys():
    prior_probability_type[i] = probability_type_count[i]/type_total
print("Type Probability: ", prior_probability_type, "\n")

################################################################################
#Find probabilities of HP, Attack, and Defense given Type

#HP Given Type
#Autocomplete used for nested if statement logic
probability_hp_gt_count = {}
for row in train_data.iterrows():
    hp = row[1]['HP']
    tt = row[1]['Type']
```

```
    if tt in probability_hp_gt_count:
        if hp in probability_hp_gt_count[tt]:
            probability_hp_gt_count[tt][hp] += 1
        else:
            probability_hp_gt_count[tt][hp] = 1
    else:
        probability_hp_gt_count[tt] = {}
        probability_hp_gt_count[tt][hp] = 1
print("HP Given Type Count: ", probability_hp_gt_count)

probability_hp_gt = {}
for tt in probability_hp_gt_count.keys():
    probability_hp_gt[tt] = {}
    for hp in probability_hp_gt_count[tt].keys():
        probability_hp_gt[tt][hp] = probability_hp_gt_count[tt][hp]/probability_typ
print("HP Given Type Probability: ", probability_hp_gt, "\n")

#Following sections autocompleted using the above code as a template
#Attack Given Type
probability_attack_gt_count = {}
for row in train_data.iterrows():
    attack = row[1]['Attack']
    tt = row[1]['Type']
    if tt in probability_attack_gt_count:
        if attack in probability_attack_gt_count[tt]:
            probability_attack_gt_count[tt][attack] += 1
        else:
            probability_attack_gt_count[tt][attack] = 1
    else:
        probability_attack_gt_count[tt] = {}
        probability_attack_gt_count[tt][attack] = 1
print("Attack Given Type Count: ", probability_attack_gt_count)

probability_attack_gt = {}
for tt in probability_attack_gt_count.keys():
    probability_attack_gt[tt] = {}
    for attack in probability_attack_gt_count[tt].keys():
        probability_attack_gt[tt][attack] = probability_attack_gt_count[tt][attack]
print("Attack Given Type Probability: ", probability_attack_gt, "\n")

#Defense Given Type
probability_defense_gt_count = {}
for row in train_data.iterrows():
    defense = row[1]['Defense']
    tt = row[1]['Type']
    if tt in probability_defense_gt_count:
        if defense in probability_defense_gt_count[tt]:
            probability_defense_gt_count[tt][defense] += 1
        else:
            probability_defense_gt_count[tt][defense] = 1
    else:
        probability_defense_gt_count[tt] = {}
        probability_defense_gt_count[tt][defense] = 1
print("Defense Given Type Count: ", probability_defense_gt_count)

probability_defense_gt = {}
```

```
for tt in probability_defense_gt_count.keys():
    probability_defense_gt[tt] = {}
    for defense in probability_defense_gt_count[tt].keys():
        probability_defense_gt[tt][defense] = probability_defense_gt_count[tt][defe
print("Defense Given Type Probability: ", probability_defense_gt, "\n")

print("\n")
##############################################################################

#Function to calculate the total number of combinations of HP, Attack, and Defense
```

In [ ]:
```
## Return probability and the class
def predict(row):
    hp = row['HP']
    attack = row['Attack']
    defense = row['Defense']

    best_score = -1
    best_class = "Unknown"

    for tt in prior_probability_type.keys():
        # Start with the prior probability of this type
        score = prior_probability_type[tt]

        # Multiply by P(HP | Type) — use 0 if this HP value was never seen in train
        score *= probability_hp_gt[tt].get(hp, 0)

        # Multiply by P(Attack | Type)
        score *= probability_attack_gt[tt].get(attack, 0)

        # Multiply by P(Defense | Type)
        score *= probability_defense_gt[tt].get(defense, 0)

        if score > best_score:
            best_score = score
            best_class = tt

    return {"score": best_score, "class": best_class}
```

In [ ]:
```

```

In [ ]:
```
### Use this loop to provide
for row in test_data.iterrows():
  print(print(row[1]["Name"]), print(predict(row[1])))
  # break
```

In [ ]:
```

```